Trees IV (Bagging and Random Forests)

Math 243

Michael Pearce

Outline

Last time, we...

- Introduced classification trees
- Implemented classification trees in R

In today's class, we will...

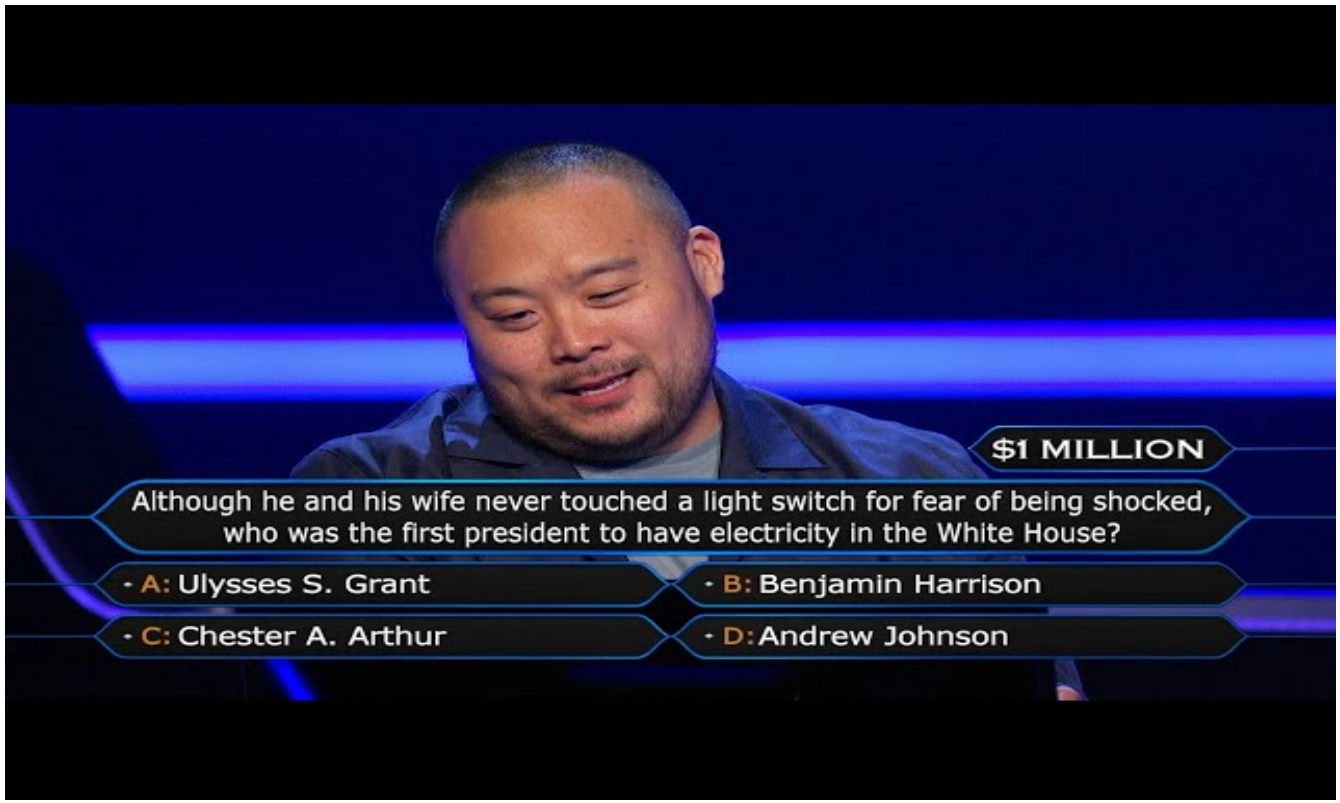
- Introduce ensemble modeling as means of improving low accuracy models
- Discuss bagging and random forests as methods for reducing variance in decision trees
- Implement random forests in R

Section 1

Ensemble Models

Who Wants to Be a Millionaire?

Who Wants to Be a Millionaire is a television gameshow in which contestants answer a series of increasingly difficult multiple choice questions in order to win the grand prize of $1,000,000.



Who Wants to Be a Millionaire?

The original show included 3 "lifeline" options contestants could use to answer questions:

- **50:50:** Two randomly selected incorrect answers are eliminated
- **Phone a Friend:** The contestant calls a friend and is given 30 seconds to discuss
- **Ask the Audience:** Audience members each vote on the answer they think is correct



Q: Which lifeline has the highest chance of producing the correct answer?

Who Wants to Be a Millionaire?

The original show included 3 "lifeline" options contestants could use to answer questions:

- **50:50:** Two randomly selected incorrect answers are eliminated
- **Phone a Friend:** The contestant calls a friend and is given 30 seconds to discuss
- **Ask the Audience:** Audience members each vote on the answer they think is correct



Q: Which lifeline has the highest chance of producing the correct answer?

- Q: Why do you think this is?

Ensemble Methods

Suppose we have m different models to predict Y based on $X_1, \ldots, X_p$.

- Let $\hat{Y}^{(i)}$ be the prediction made by the ith model

Ensemble Methods

Suppose we have m different models to predict Y based on $X_1, \ldots, X_p$.

- Let $\hat{Y}^{(i)}$ be the prediction made by the ith model

An Ensemble Model makes a prediction $\hat{Y}$ as the weighted average of the predictions from each model:

$$\hat{Y} = \frac{1}{m}\hat{Y}^{(1)} + \cdots + \frac{1}{m}\hat{Y}^{(m)}$$

Ensemble Methods

Suppose we have m different models to predict Y based on $X_1, \ldots, X_p$.

- Let $\hat{Y}^{(i)}$ be the prediction made by the ith model

An Ensemble Model makes a prediction $\hat{Y}$ as the weighted average of the predictions from each model:

$$\hat{Y} = \frac{1}{m}\hat{Y}^{(1)} + \cdots + \frac{1}{m}\hat{Y}^{(m)}$$

Q: What are some advantages of ensemble models? Disadvantages?

Ensemble Methods

Suppose we have m different models to predict Y based on $X_1, \ldots, X_p$.

- Let $\hat{Y}^{(i)}$ be the prediction made by the ith model

An Ensemble Model makes a prediction $\hat{Y}$ as the weighted average of the predictions from each model:

$$\hat{Y} = \frac{1}{m}\hat{Y}^{(1)} + \cdots + \frac{1}{m}\hat{Y}^{(m)}$$

Q: What are some advantages of ensemble models? Disadvantages?

Some Advantages:

- More flexible than a single model
- Lower variance than a single model
- More resilient against bias of any specific model

Some Disadvantages:

- Making predictions is more computationally expensive
- Implicitly favors models with low time/computational cost
- Diminishing returns as more models are incorporated in ensemble

Section 2

Bagging

Bagging

Suppose we want to build an ensemble of **regression tree** models.

- Problem: We only have one training set..

Bagging

Suppose we want to build an ensemble of **regression tree** models.

- Problem: We only have one training set..
- Idea: Make a bunch of "new" training sets by bootstrapping!

Bagging

Suppose we want to build an ensemble of **regression tree** models.

- Problem: We only have one training set..
- Idea: Make a bunch of "new" training sets by bootstrapping!

Bagging (**B**ootstrap **agg**regation) – an early ensemble technique:

1. Create many bootstrap samples from the original training data set
2. Fit a tree to each bootstrap sample
3. Average predictions across trees

Bagging

Suppose we want to build an ensemble of **regression tree** models.

- Problem: We only have one training set..
- Idea: Make a bunch of "new" training sets by bootstrapping!

Bagging (**B**ootstrap **agg**regation) – an early ensemble technique:

1. Create many bootstrap samples from the original training data set
2. Fit a tree to each bootstrap sample
3. Average predictions across trees

Why?

- Trees tend to have high variance, but averaging models decreases variance
- Unlike a single tree, we **do not prune**. Instead, reduce variance by averaging

Out-of-Bag Error: Estimated Test Error for free!

Recall (from homework): An individual observation has probability $e^{-1} = 0.368$ of **not** appearing in a bootstrap sample

- So for each bootstrap sample, approximately $1/3$ of observations are **not** included – we call these Out-of-Bag observations

Out-of-Bag Error: Estimated Test Error for free!

Recall (from homework): An individual observation has probability $e^{-1} = 0.368$ of **not** appearing in a bootstrap sample

- So for each bootstrap sample, approximately 1/3 of observations are **not** included – we call these Out-of-Bag observations

Out-of-bag observations can be used as a natural validation set for the bootstrap model

- We get an overall estimate of test MSE for the bagged model by averaging the MSE of each bootstrap model on its out-of-bag observations

A Bag of Trees

We return to the pdxTrees data set, this time expanding both our data set size and number of predictors:

```
names(my_pdxTrees)
```

```
## [1] "DBH"                    "Condition"
## [3] "Tree_Height"            "Crown_Width_NS"
## [5] "Crown_Width_EW"         "Crown_Base_Height"
## [7] "Functional_Type"        "Mature_Size"
## [9] "Carbon_Sequestration_lb"
```

```
dim(my_pdxTrees)
```

```
## [1] 3015    9
```

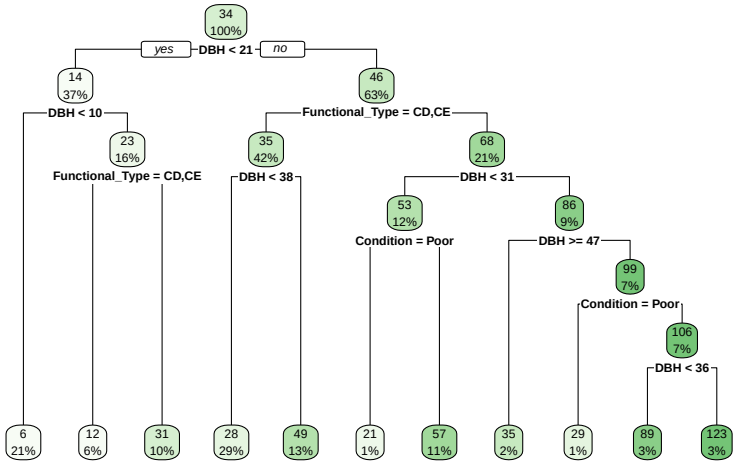
Goal: Improve previous model predicting Carbon_Sequestration_lb, now using **more data** and **more predictors!**

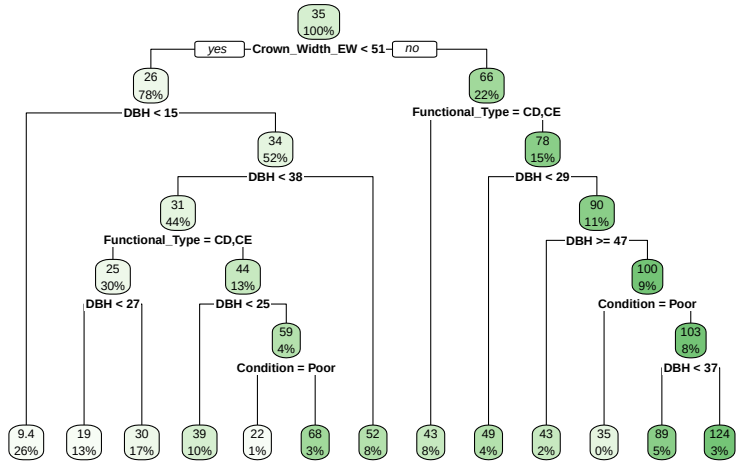
Bagged pdxTrees

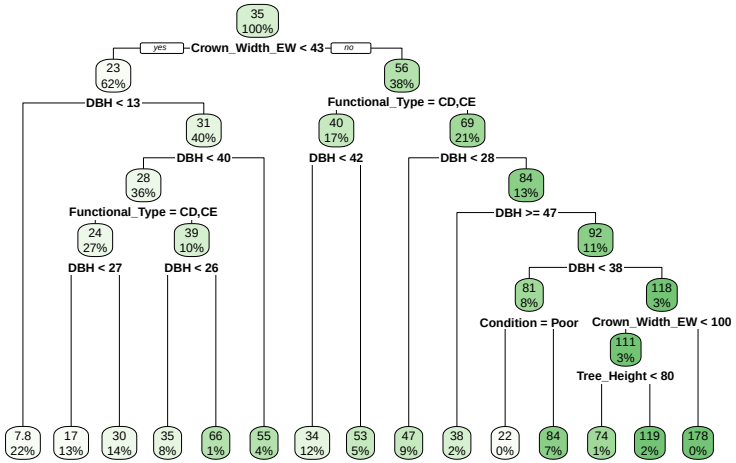
For a small demonstration, I will...

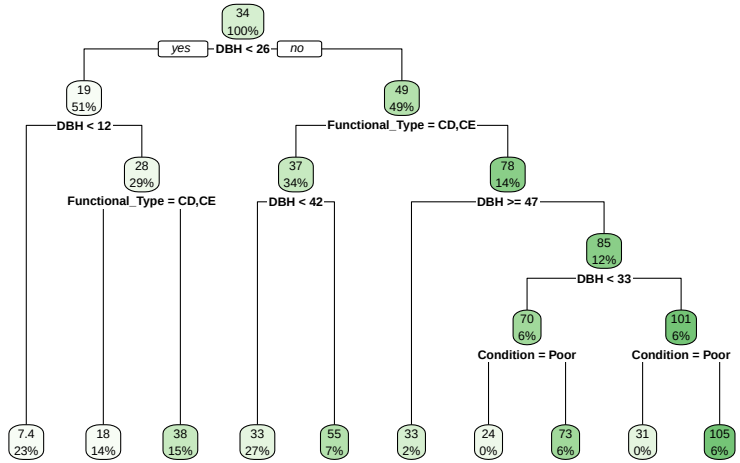
1. Create **4 bootstrap** samples from the training data
2. For each, fit a tree model **without pruning**
3. Calculate the RMSE for **each model** on the test data.
4. Calculate the RMSE for the **bagged** model that averages the four trees.

Trees fit to 4 bootstrap samples



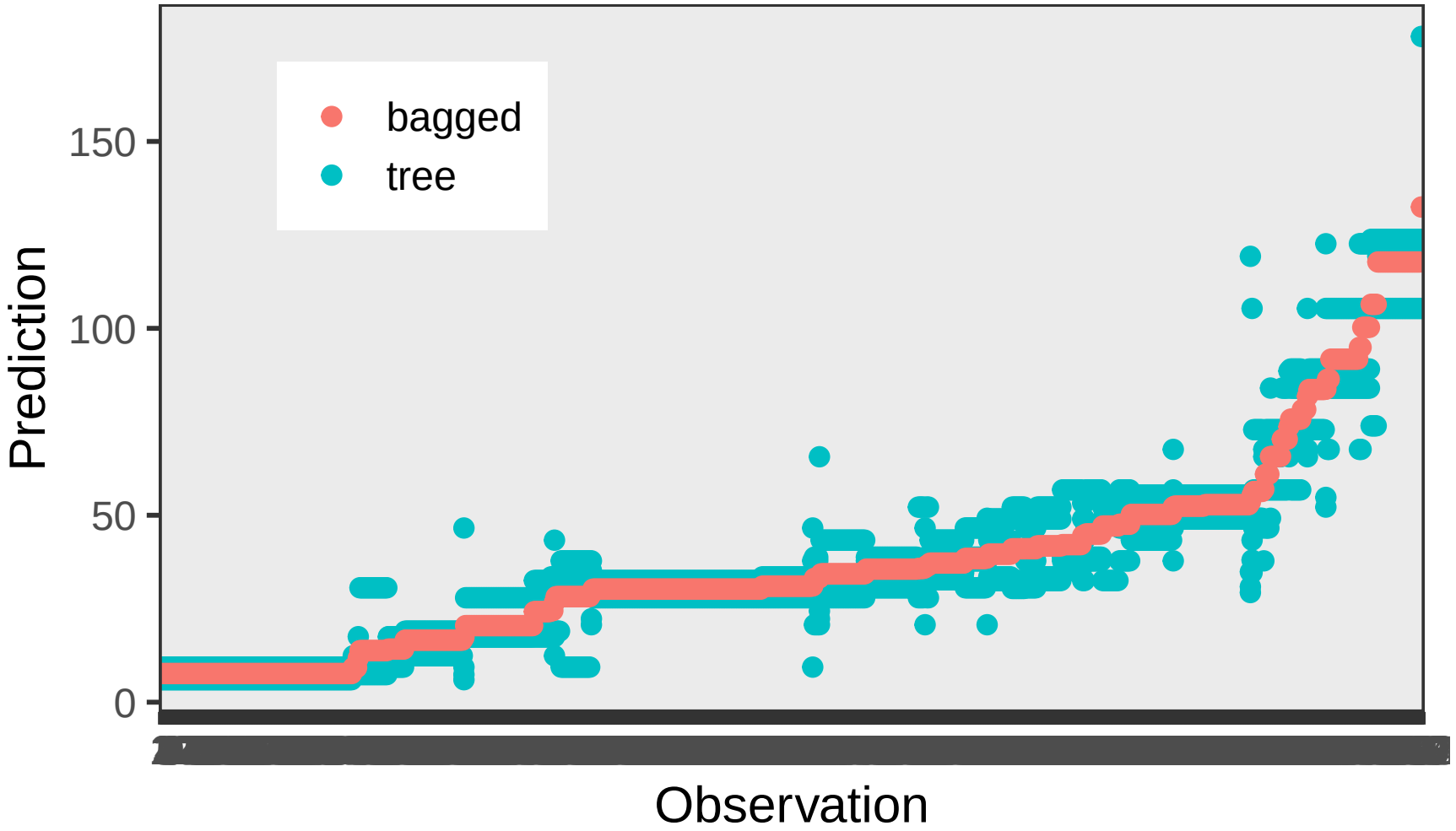






Test Set Performance

First, we compare predictions made by individual trees and the bagged model across observations:



Test Set Performance

Second, we display overall RMSE for each tree model and the Bagged Tree:

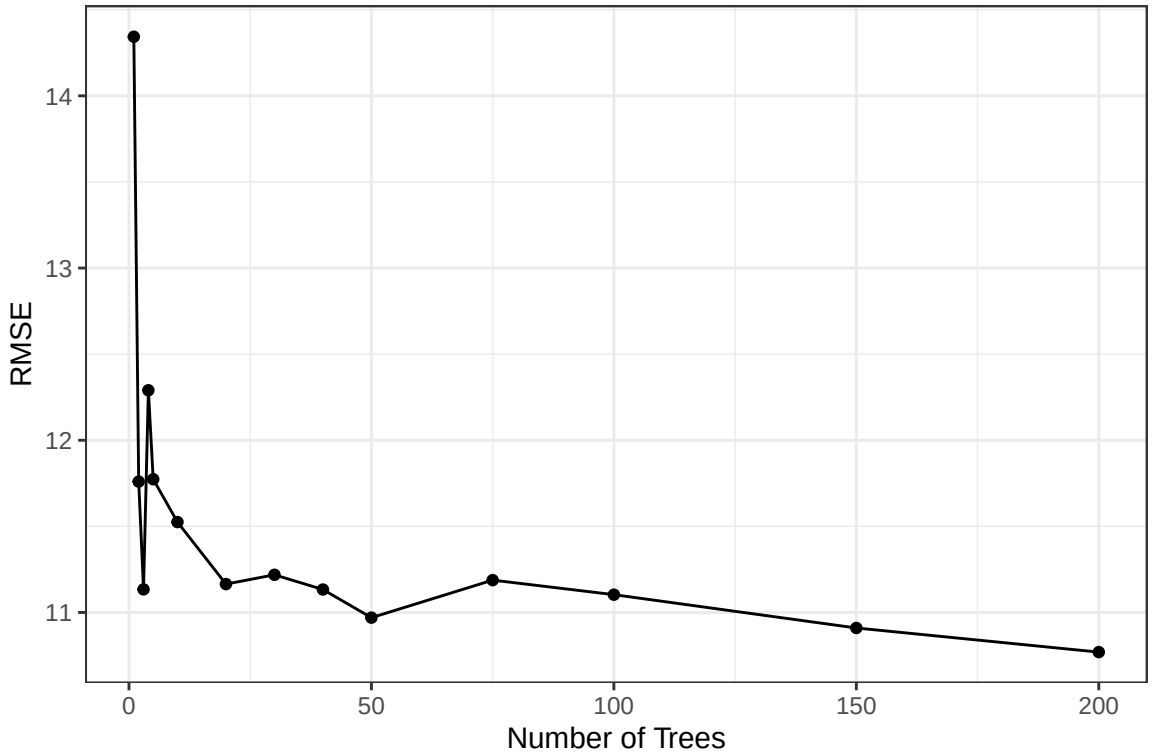
```
## # A tibble: 5 x 3
##   model  .metric .estimate
##   <chr>  <chr>       <dbl>
## 1 tree 1 rmse         14.0
## 2 tree 2 rmse         14.3
## 3 tree 3 rmse         14.3
## 4 tree 4 rmse         13.1
## 5 bagged rmse         12.3
```

The more trees the merrier?

If 4 trees improved performance over 1, what if we bagged 10 trees? 100?

The more trees the merrier?

If 4 trees improved performance over 1, what if we bagged 10 trees? 100?



- Greatest gains by adding a small number of additional trees
- Moderately small gains thereafter

Section 3

Random Forests

Further Performance Improvements

Suppose we have m ensemble models built from the same data set and it turns out that all models are very similar (i.e., highly correlated)

- Q: Would we still expect the ensemble model to have much lower variance than any individual model?

Further Performance Improvements

Suppose we have m ensemble models built from the same data set and it turns out that all models are very similar (i.e., highly correlated)

- Q: Would we still expect the ensemble model to have much lower variance than any individual model?
- Probably not!

Further Performance Improvements

Suppose we have m ensemble models built from the same data set and it turns out that all models are very similar (i.e., highly correlated)

- Q: Would we still expect the ensemble model to have much lower variance than any individual model?
- Probably not!

Bagging helps, but it turns out that the trees are often pretty similar (i.e., correlated)

- If one predictor is very predictive of the response, it will be used for the first split in most of the bootstrap samples

Further Performance Improvements

Suppose we have m ensemble models built from the same data set and it turns out that all models are very similar (i.e., highly correlated)

- Q: Would we still expect the ensemble model to have much lower variance than any individual model?
- Probably not!

Bagging helps, but it turns out that the trees are often pretty similar (i.e., correlated)

- If one predictor is very predictive of the response, it will be used for the first split in most of the bootstrap samples

Crazy idea: To increase the variety among trees, **Random Forests** randomly restrict which predictors can be used at each *split* point when growing a tree

- Counter-intuitive, but this tends to increase the accuracy of the ensemble by breaking correlations among the participant trees

Random Forests

Random Forest:

1. Select the number of models m to build (from m bootstrap samples) and a number of predictors k to use at each step of tree-building process t

Random Forests

Random Forest:

1. Select the number of models m to build (from m bootstrap samples) and a number of predictors k to use at each step of tree-building process t
2. Generate a bootstrap sample for each model
3. Grow a tree on each bootstrap sample where at each split, a random selection of k of the p predictors can be used

Random Forests

Random Forest:

1. Select the number of models m to build (from m bootstrap samples) and a number of predictors k to use at each step of tree-building process t
2. Generate a bootstrap sample for each model
3. Grow a tree on each bootstrap sample where at each split, a random selection of k of the p predictors can be used
4. Aggregate (i.e., average) the m models to create the ensemble model.

Random Forests

Random Forest:

1. Select the number of models m to build (from m bootstrap samples) and a number of predictors k to use at each step of tree-building process t
2. Generate a bootstrap sample for each model
3. Grow a tree on each bootstrap sample where at each split, a random selection of k of the p predictors can be used
4. Aggregate (i.e., average) the m models to create the ensemble model.

Advantages:

- Individual models are less correlated, so ensemble has lower variance
- Each tree is quicker to build (Q: Why?)

Disadvantages:

- Difficult to interpret
- Theoretical properties less well-studied

Section 4

Bagging and Random Forests in R

Random Forest in R with `randomForest`

To create both bagged trees and random forests, we use the `randomForest` function in the `randomForest` package in R:

Random Forest in R with `randomForest`

To create both bagged trees and random forests, we use the `randomForest` function in the `randomForest` package in R:

```
library(randomForest)
rfmodel <- randomForest(Carbon_Sequestration_lb ~ ., data = my_pdxTrees_train)
rfmodel
```

```
##
## Call:
##  randomForest(formula = Carbon_Sequestration_lb ~ ., data = my_pdxTrees_train)
##                Type of random forest: regression
##                      Number of trees: 500
## No. of variables tried at each split: 2
##
##           Mean of squared residuals: 112.2864
##                     % Var explained: 85.74
```

- Code structure very similar to `lm()` function, but you can further modify how the random forest is built...

Modifications: `ntree` and `mtry`

We modify the options:

- `ntree`: how many trees are generated
- `mtry`: the number of predictors considered at each split

```
set.seed(1)
rfmodel2 <- randomForest(Carbon_Sequestration_lb ~ ., data = my_pdxTrees_train,
                         ntree = 10, mtry = 5)
rfmodel2
```

```
##
## Call:
##  randomForest(formula = Carbon_Sequestration_lb ~ ., data = my_pdxTrees_train,      ntree = 10, mtry = 5)
##                Type of random forest: regression
##                      Number of trees: 10
## No. of variables tried at each split: 5
##
##           Mean of squared residuals: 106.4475
##                     % Var explained: 86.48
```

Modifications: `ntree` and `mtry`

We modify the options:

- `ntree`: how many trees are generated
- `mtry`: the number of predictors considered at each split

```
set.seed(1)
rfmodel2 <- randomForest(Carbon_Sequestration_lb ~ ., data = my_pdxTrees_train,
                         ntree = 10, mtry = 5)
rfmodel2
```

```
##
## Call:
##  randomForest(formula = Carbon_Sequestration_lb ~ ., data = my_pdxTrees_train,      ntree = 10, mtry = 5)
##                Type of random forest: regression
##                      Number of trees: 10
## No. of variables tried at each split: 5
##
##           Mean of squared residuals: 106.4475
##                     % Var explained: 86.48
```

By default, `randomForest` makes 500 trees, uses $p/3$ predictors for regression and $\sqrt{p}$ predictors for classification.

Q: How can we create a *bagging* ensemble model using the `randomForest` function?

Modifications: `ntree` and `mtry`

We modify the options:

- `ntree`: how many trees are generated
- `mtry`: the number of predictors considered at each split

```
set.seed(1)
rfmodel2 <- randomForest(Carbon_Sequestration_lb ~ ., data = my_pdxTrees_train,
                         ntree = 10, mtry = 5)
rfmodel2
```

```
##
## Call:
##  randomForest(formula = Carbon_Sequestration_lb ~ ., data = my_pdxTrees_train,      ntree = 10, mtry = 5)
##                Type of random forest: regression
##                      Number of trees: 10
## No. of variables tried at each split: 5
##
##           Mean of squared residuals: 106.4475
##                     % Var explained: 86.48
```

By default, `randomForest` makes 500 trees, uses $p/3$ predictors for regression and $\sqrt{p}$ predictors for classification.

Q: How can we create a *bagging* ensemble model using the `randomForest` function?

- Set `mtry = p`, where `p` is the total number predictors available

Making predictions with `predict()`

You can make predictions using the `predict()` function:

```
predict(rfmodel, my_pdxTrees_test)
```

```
##        1        2        3        4        5
## 38.85781 66.09302 75.53011 33.41863 51.02538
```

Making predictions with `predict()`

You can make predictions using the `predict()` function:

```
predict(rfmodel, my_pdxTrees_test)
```

```
##        1        2        3        4        5
## 38.85781 66.09302 75.53011 33.41863 51.02538
```

Calculate RMSE "by hand"

```
residuals <- predict(rfmodel,my_pdxTrees_test) - my_pdxTrees_test$Carbon_Sequestrati
RMSE <- sqrt( mean( residuals^2 ) )
RMSE
```

```
## [1] 11.35972
```

Variable Importance

Bagging and Random Forests increase prediction accuracy by reducing variance of the model.

- But they are far less interpretable.
- We no longer have a single decision tree to follow to reach our prediction.

Variable Importance

Bagging and Random Forests increase prediction accuracy by reducing variance of the model.

- But they are far less interpretable.
- We no longer have a single decision tree to follow to reach our prediction.

Variable Importance helps show which predictors are most influential

- Variable importance records the improvement in RSS/Purity due to splits on a given predictor from each tree

Variable Importance

Bagging and Random Forests increase prediction accuracy by reducing variance of the model.

- But they are far less interpretable.
- We no longer have a single decision tree to follow to reach our prediction.

Variable Importance helps show which predictors are most influential

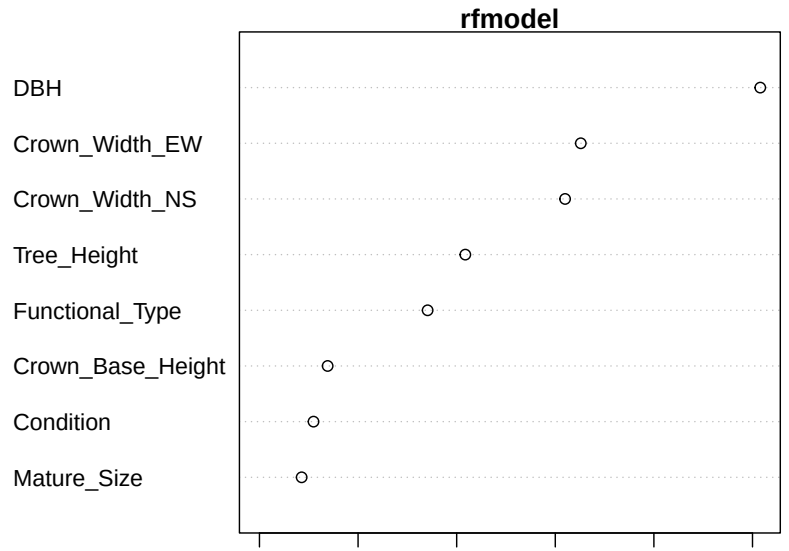
- Variable importance records the improvement in RSS/Purity due to splits on a given predictor from each tree
- These "improvements" are then averaged across all trees in the random forest

Variable Importance in R

```
importance(rfmodel)
```

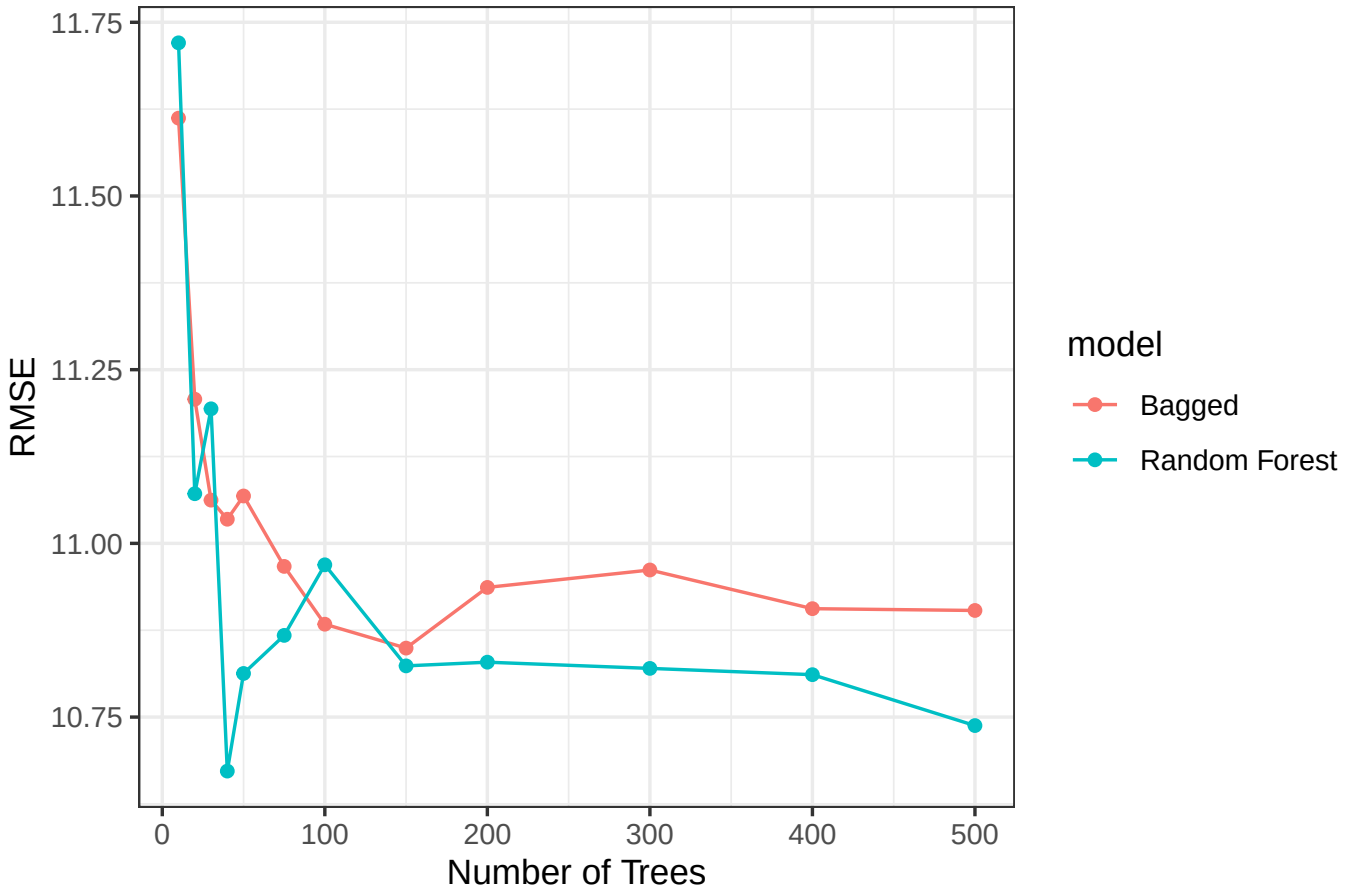
```
##                    IncNodePurity
## DBH                    507862.05
## Condition               54821.17
## Tree_Height            208750.74
## Crown_Width_NS         309930.48
## Crown_Width_EW         325846.81
## Crown_Base_Height       69137.26
## Functional_Type        170538.17
## Mature_Size             42785.73
```

```
varImpPlot(rfmodel)
```



- For regression trees, `IncNodePurity` is calculated using RSS.
- For classification trees, `IncNodePurity` is calculated using the Gini Index

Comparison of Bagged Trees versus Random Forests



- Random forests are (often) better than just bagging!

Section 5

Recap

Recap

Today, we...

- Introduced ensembled methods
- Learned about Bagging, which is a special case of Random Forests
- Implemented Bagging and Random Forests in R

Next time, we will...

- Practice Classification Trees, Bagging, and Random Forests in a lab.
- Next week: Another tree method: "Boosting"